

GEOSPATIAL PLACE NAME EXTRACTOR & ENTITY NORMALIZER

A Data Structures and Algorithms Project Report

C-based geographic entity detection, normalization and spelling correction



**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY
AND SCIENCES**

CERTIFICATE

This is to certify that the project report entitled "Rate Limiter Lab: A Stateful Backend Engine and Interactive Visualizer for Five Rate-Limiting Algorithms" submitted by the following students of HI/IV B.Tech - CSE, Academic Year 2026-2027, is a bonafide record of the work carried out by them under my supervision and guidance.

A25126510081 E.V.S.Bharath Pramodh
A25126510094 M.R.S Praneeth
A25126510098 P.Stephen
A25126510107 P.Harishith
A25126510126 T.Sai Purna Chand

The report has been submitted in partial fulfillment of the requirements for the course **Data Structures and Algorithms** (Semester II, and has not been submitted to any other university or institution for the award of any degree or diploma.

Dr. B. Meena
Faculty, Data Structures and Algorithms ANITS,
Visakhapatnam

Head of Department Department
of CSE ANITS, Visakhapatnam

Signature _____

Signature _____

Date:
Place: Visakhapatnam

DECLARATION

I hereby declare that the project titled “Geospatial Place Name Extractor & Entity Normalizer” is an academic implementation developed to demonstrate the practical use of Data Structures and Algorithms.

The system is written in C and uses a GeoNames geographic data extract as its geographic knowledge source. The implementation includes data extraction, filtering, in-memory storage, exact matching, alternate-name matching, common-word filtering, fuzzy spelling correction and sentence reconstruction.

The project has been developed as an educational demonstration of algorithmic problem solving on a large geographic dataset.

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty members and project guide for providing the opportunity to work on this project.

This project helped connect theoretical DSA concepts with a practical problem involving millions of geographic records. I am thankful for the guidance that helped in understanding file parsing, structures, searching, string processing and edit-distance algorithms.

I also acknowledge the GeoNames project for providing a worldwide geographic database extract. The project uses this data as the geographic gazetteer.

Finally, I acknowledge everyone who supported the testing and debugging stages, especially while solving empty-field parsing, false-positive detection, alternate-name ranking and spelling-correction problems.

ABSTRACT

The Geospatial Place Name Extractor & Entity Normalizer is a C-based Data Structures and Algorithms project that identifies geographic place names inside natural-language sentences and maps them to normalized canonical names.

The system uses the GeoNames worldwide gazetteer. The project downloads and extracts the worldwide data, parses its tab-separated records, filters useful geographic entities and produces a smaller project-specific locations.txt file.

The application performs exact matching for canonical names and alternate names. A ranking mechanism uses geographic feature priority and population to choose between competing candidates. A common-word filter prevents ordinary words such as “from”, “to”, “and” and “travel” from being falsely interpreted as locations.

When exact matching fails, the system uses Levenshtein edit distance to identify a close spelling candidate. For example, a misspelled form such as “mumbal” can be mapped to “Mumbai”. The canonical name is then inserted into the complete sentence.

During the documented project run, 13,472,285 raw records were read and 5,775,721 useful locations were retained. The project therefore demonstrates a complete pipeline from large geographic data preprocessing to sentence-level entity normalization.

The project also establishes a foundation for future DSA improvements such as hash tables, tries, BK-trees, multi-word location detection and context-aware disambiguation.

TABLE OF CONTENTS / INDEX

1. AIM
2. PROBLEM STATEMENT
3. INTRODUCTION
4. OBJECTIVES OF THE PROJECT
5. SCOPE OF THE PROJECT
6. LITERATURE SURVEY
7. EXISTING AND PROPOSED SYSTEM
8. DATASET AND GEONAMES
9. TOOLS / SOFTWARE AND HARDWARE REQUIREMENTS
10. SYSTEM ARCHITECTURE AND METHODOLOGY
11. DATA PREPROCESSING AND EXTRACTION
12. DATA STRUCTURES AND ALGORITHMS USED
13. IMPLEMENTATION DETAILS
14. PROJECT WORKFLOW
15. TESTING AND TEST CASES
16. RESULTS AND DISCUSSION
17. COMPLEXITY ANALYSIS
18. LIMITATIONS AND CHALLENGES
19. FUTURE ENHANCEMENTS
20. CONCLUSION AND LEARNING OUTCOMES
21. REFERENCES / BIBLIOGRAPHY
22. APPENDIX

The primary aim of this project is to develop a C-based geospatial place-name extraction and entity-normalization system using Data Structures and Algorithms. The system is designed to identify geographic place names in natural-language sentences and replace alternate or misspelled forms with appropriate canonical names.

The project aims to combine file processing, structured data storage, string matching, searching, ranking and dynamic programming in one practical application. It uses the GeoNames worldwide gazetteer as the geographic knowledge source.

The final output is a corrected sentence in which recognized geographic entities are normalized to canonical names while ordinary words, separators and the remaining sentence content are preserved.

- Read and filter a large worldwide geographic dataset.
- Store useful geographic records in memory.
- Recognize canonical and alternate place names.
- Reduce false positives using common-word filtering.
- Correct reasonable spelling errors using Levenshtein distance.
- Rank competing geographic candidates using feature priority and population.
- Generate a normalized complete sentence.

2. PROBLEM STATEMENT

Design and implement a C program that accepts a sentence containing geographic place names, identifies those places, recognizes alternate names, corrects reasonable spelling mistakes and outputs a normalized sentence using canonical geographic names.

2.1 Motivation

Exact lookup works only when the user types the expected spelling. Real users may use alternate names such as Bombay or make small errors such as mumbal. The project therefore needs both exact and approximate matching.

2.2 Main Challenges

Millions of geographic records.

Empty fields in tab-separated source data.

Multiple alternate names.

Common English words that resemble place names.

Duplicate or ambiguous geographic names.

Spelling errors.

Preserving the original sentence structure.

2.3 DSA Perspective

The problem gives a practical context for arrays, structures, string comparison, linear searching, ranking rules and dynamic programming.

3. INTRODUCTION

Geographic names occur in travel messages, news, directions, descriptions and many other forms of text. The same geographic entity may be written using its canonical name, an alternate name or a misspelled form.

This project builds a geographic place-name extractor and entity normalizer in C. Given a sentence such as “iam travelling from bombay to bangalore”, the system identifies the geographic terms and maps them to canonical names.

3.1 Core Idea

- Read a worldwide geographic dataset.
- Filter it into useful geographic entities.
- Load the records into memory.
- Check sentence words for exact canonical and alternate matches.
- Ignore ordinary common English words.
- Use Levenshtein distance for reasonable spelling errors.
- Reconstruct the complete corrected sentence.

3.2 Academic Relevance

The project is not simply a geographic lookup. It combines file handling, structures, arrays, string algorithms, searching, ranking and dynamic programming in one practical application.

3.3 Expected Output

The expected output is a complete sentence in which recognized locations are represented using their selected canonical geographic names.

4. OBJECTIVES OF THE PROJECT

The main objective is to design and implement a practical C program that can extract geographic entities from text and normalize them reliably using DSA techniques.

Build a worldwide geographic dataset.

Filter useful geographic records.

Implement exact canonical matching.

Implement alternate-name matching.

Use feature priority and population for candidate ranking.

Reduce common-word false positives.

Implement Levenshtein fuzzy matching.

Generate a corrected complete sentence.

Demonstrate practical Data Structures and Algorithms concepts.

4.1 Specific Functional Objectives

- Preserve empty fields while parsing tab-separated records.
- Load the filtered location collection into memory.
- Compare input words independently of letter case.
- Support canonical and alternate geographic names.
- Apply length filtering before expensive fuzzy comparisons.
- Preserve separators and non-location words during sentence reconstruction.

5. SCOPE OF THE PROJECT

The current project focuses on countries, administrative entities and populated places selected from GeoNames. GeoNames defines administrative feature codes including ADM1 through ADM5 and populated-place codes including PPL, PPLA and PPLC.

5.1 What Is Covered

- Worldwide geographic data preparation.
- Filtering of useful geographic records.
- Canonical and alternate-name recognition.
- Common-word filtering.
- Feature-priority and population-based ranking.
- Levenshtein-based spelling correction.
- Sentence-level normalization.
- Testing and result generation.

5.2 Current Exclusions

- Full business or point-of-interest recognition.
- Guaranteed multi-word location recognition.
- Complete contextual disambiguation.
- Map display and route calculation.
- Machine-learning NER.
- Automatic live database updates.

5.3 Practical Applications

- Text normalization involving geographic names.
- Educational demonstrations of DSA.
- Geographic search preprocessing.
- Data-cleaning workflows for location names.
- Foundation for larger geographic information systems.

6. LITERATURE SURVEY

The project is based on established ideas in geographic information processing, string matching, information retrieval and Data Structures and Algorithms. The central requirement is to map different textual forms of a place to one canonical geographic entity.

6.1 Exact Lookup

Basic geographic lookup stores a list of place names and compares an input against the stored values. It is simple to understand but does not scale naturally to worldwide data and cannot adequately handle alternate names or spelling errors.

6.2 Alternate-Name Normalization

A geographic entity can have more than one commonly used name. Alternate-name support maps multiple surface forms to one canonical entity. In the project, Bombay is treated as an alternate form that can normalize to Mumbai.

6.3 Approximate String Matching

Levenshtein distance is used to measure the minimum number of single-character insertions, deletions and substitutions needed to transform one string into another. This makes it suitable for correcting small spelling errors.

6.4 Ranking and Disambiguation

When the same input can correspond to multiple geographic records, deterministic ranking is required. The project uses feature priority and population as selection criteria.

6.5 Research Gap Addressed

- Small lookup lists do not scale to worldwide geographic data.
- Simple exact matching does not handle alternate names or spelling errors.
- Naive word-by-word lookup can produce false positives.
- Ambiguous geographic names require deterministic candidate ranking.

7. EXISTING AND PROPOSED SYSTEM

7.1 Existing / Basic Approach

A basic program can store a small list of cities and compare the input with each item. Such a design is easy to understand but does not scale well to worldwide data and cannot naturally handle alternate names or spelling errors.

Requirement	Basic Lookup	Proposed Project
Exact names	Yes	Yes
Alternate names	Limited	Yes
Spelling errors	No	Yes
Worldwide data	Difficult	Yes
Common-word filtering	No	Yes
Candidate ranking	Limited	Yes
Sentence correction	Limited	Yes

7.2 Proposed System

The proposed system separates data preparation from sentence processing. Geographic data is first parsed and filtered. The resulting records are loaded into memory. Each sentence word is then processed through common-word filtering, exact matching, fuzzy matching and normalization.

7.3 Main Advantages

- Clear DSA pipeline.
- Worldwide geographic knowledge source.
- Support for canonical and alternate names.
- Protection against demonstrated common-word false positives.
- Spelling correction for reasonable errors.
- Deterministic candidate ranking.

8. DATASET AND GEONAMES

8.1 GeoNames

GeoNames provides a worldwide geographic database and an official export service. The project uses the worldwide geographic extract as its geographic knowledge source.

8.2 Fields Used

Field	Purpose
Name	Canonical place name
Alternate Names	Alternative forms
Feature Class	Broad geographic classification
Feature Code	Detailed geographic type
Country	Country association
Admin1	First-level administrative association
Population	Candidate tie-breaking
Latitude / Longitude	Geographic coordinates retained in extraction

8.3 Feature Filtering

The project retains Feature Class A records and selected Feature Class P records. For Feature Class P, PCLI and PPL-prefixed records are accepted.

8.4 Important Feature Codes

Code	Meaning
ADM1	First-order administrative division
PPL	Populated place
PPLA	Seat of a first-order administrative division
PPLC	Capital of a political entity

9. TOOLS / SOFTWARE AND HARDWARE REQUIREMENTS

9.1 Hardware Requirements

- Modern laptop or desktop.
- Adequate RAM for the large in-memory location array.
- Several GB of free storage.
- Keyboard and display.

9.2 Software Requirements

Component	Purpose
macOS / compatible OS	Execution environment
VS Code	Editing
Terminal	Build and run
GCC / Clang	C compiler
GeoNames data	Geographic knowledge source

9.3 Project Directory

Geospatial-place-extactor/

- bin/
- data/ → allCountries.zip, allCountries.txt, locations.txt
- src/ → geospatial_extractor.c, location_search.c, sentence_processor.c, location_index.c

9.4 Development Environment

The integrated sentence processor is the main application. The other modules were used during staged development and testing.

10. SYSTEM ARCHITECTURE AND METHODOLOGY

10.1 High-Level Flow

GeoNames allCountries.zip → allCountries.txt → C preprocessing and filtering → locations.txt → Load Location records → User sentence → Exact / Fuzzy Search → Canonical location → Corrected sentence

10.2 Modules

Module	Responsibility
Extraction	Parse and filter raw GeoNames data
Loading	Read locations.txt
Exact search	Canonical and alternate lookup
Filtering	Common-word protection
Fuzzy search	Spelling correction
Ranking	Feature priority and population
Sentence processor	Replace detected locations and preserve text

10.3 Data Flow

The main design principle is a one-way processing pipeline: raw data is transformed into a usable location collection, and sentence input is transformed into normalized text.

11. DATA PREPROCESSING AND EXTRACTION

11.1 Raw Data Parsing

GeoNames records are tab-separated. During development, using a delimiter tokenizer that removes empty fields caused field positions to shift. A custom `split_tab()` routine was therefore implemented.

11.2 `split_tab()`

The function scans each line, converts tabs into string terminators and stores field starting positions. Consecutive tabs are preserved, so empty fields remain represented.

11.3 Generated `locations.txt`

- ID
- Name
- ASCII Name
- Alternate Names
- Latitude
- Longitude
- Feature Class
- Feature Code
- Country
- Admin1
- Population

11.4 Observed Dataset Statistics

Metric	Observed Value
Raw records read	13,472,285
Useful locations	5,775,721
Configured maximum	6,000,000

These values describe the documented project run and may differ if a newer GeoNames extract is used.

12. DATA STRUCTURES AND ALGORITHMS USED

12.1 Location Structure

The Location structure groups the fields needed by the search and normalization stages.

```
typedef struct { char name[NAME_SIZE]; char alternateNames[ALT_NAME_SIZE]; char  
featureCode[20]; long population; } Location;
```

12.2 Arrays

- Character arrays store names and sentences.
- fields[] stores parsed source fields.
- locations[] stores loaded geographic records.

12.3 Dynamic Memory

malloc() is used to allocate the large location collection at runtime, and free() releases it after processing.

12.4 Strings

String processing is central to the application. A custom case-insensitive comparison function makes matching independent of letter case.

12.5 Exact Search

Receive the sentence word.

Check common-word status.

Scan loaded locations.

Compare canonical name.

If needed, compare alternate names.

Rank matching candidates.

Return the selected canonical name.

13. IMPLEMENTATION DETAILS

13.1 Main Functions

Function	Purpose
toLowerChar	Lowercase conversion
caseInsensitiveCompare	Case-insensitive equality
split_tab	Preserve empty fields
isUsefulLocation	Dataset filtering
loadLocations	Load records
getFeaturePriority	Rank features
findExactAlternate	Alternate-name lookup
searchLocation	Exact search
isCommonWord	False-positive protection
levenshteinDistance	Edit distance
findFuzzyLocation	Approximate search
processSentence	Sentence normalization

13.2 Compile and Run

```
cd /Users/saipurnachand/Geospatial-place-extactor
gcc src/sentence_processor.c -o bin/sentence_processor
./bin/sentence_processor
```

13.3 Error Handling

- File-open failure is detected.
- Memory allocation failure is detected.
- Empty input is handled.
- Maximum location count is protected.
- Temporary Levenshtein allocations are checked.

14. PROJECT WORKFLOW

14.1 Development Pipeline

Obtain GeoNames worldwide data.

Extract allCountries.txt.

Parse tab-separated records.

Filter geographic features.

Generate locations.txt.

Test exact search.

Add alternate names.

Add common-word filtering.

Add Levenshtein matching.

Reconstruct corrected sentence.

Test the complete system.

14.2 Problems Solved During Development

- Empty-field parsing.
- False location detection.
- Incorrect alternate-name ranking.
- Spelling correction.
- Full-sentence reconstruction.

14.3 Current Status

The integrated program loads the filtered location collection and supports exact canonical names, alternate names, common-word filtering and fuzzy spelling correction.

15. TESTING AND TEST CASES

15.1 Test Strategy

Testing was performed incrementally after each major feature. This made it possible to identify parsing errors, matching errors and ranking errors independently.

ID	Input	Expected Result
T1	mumbai	Mumbai — exact match
T2	bombay	Mumbai — alternate normalization
T3	bangalore	Bengaluru — canonical result
T4	mumbal	Mumbai — fuzzy candidate
T5	hyderabad	Hyderabad — fuzzy candidate
T6	from	Ignored as common word
T7	to	Ignored as common word
T8	bombay to bangalore	Only places detected
T9	mumbal to banglore	Misspellings corrected
T10	empty	Input warning

15.2 Negative Testing

The common-word filter is specifically tested because a geographic gazetteer can contain place names that look like ordinary words.

16. RESULTS AND DISCUSSION

16.1 Dataset Result

During the documented project run, 13,472,285 raw records were read and 5,775,721 useful locations were retained, against a configured maximum of 6,000,000 records.

16.2 Exact Matching Result

Input: iam travelling from mumbai to bangalore

Detected: mumbai → Mumbai; bangalore → Bengaluru

16.3 Alternate Matching Result

Input: iam travelling from bombay to bangalore

Detected: bombay → Mumbai; bangalore → Bengaluru

16.4 Fuzzy Matching Result

Input: iam travelling from mumbal to banglore

Detected: mumbal → Mumbai (distance: 1); banglore → Bengaluru (distance: 1)

Corrected sentence: iam travelling from Mumbai to Bengaluru

17. COMPLEXITY ANALYSIS

17.1 Loading

If N is the number of retained records, loading requires $O(N)$ record processing, in addition to file I/O.

17.2 Exact Search

The current linear array implementation requires $O(N)$ worst-case time for one exact query. For W candidate words, repeated searches can approach $O(WN)$.

17.3 Levenshtein Distance

For strings of lengths m and n , dynamic-programming Levenshtein distance requires $O(mn)$ time. The two-row implementation uses $O(n)$ auxiliary memory.

17.4 Fuzzy Search

The fuzzy search scans the location collection. The length filter reduces the number of Levenshtein calculations, but worst-case search still depends on N .

17.5 Space

The current Location structure uses fixed-size character arrays, especially a large alternateNames field. This makes the in-memory representation memory intensive.

18. LIMITATIONS AND CHALLENGES

18.1 Current Limitations

- Large memory usage due to fixed-size record fields.
- Linear exact search.
- Potentially slow fuzzy search.
- Single-word-oriented sentence tokenization.
- No dedicated multi-word place recognition.
- Manually maintained common-word list.
- Heuristic rather than full contextual disambiguation.
- GeoNames does not guarantee every commercial map POI.

18.2 Major Development Challenges

Empty Fields: Empty tab-separated fields initially caused incorrect field positions. The custom parser fixed this.

False Positives: Normal words such as from and to were incorrectly detected as places. The common-word filter fixed the demonstrated cases.

Alternate Ranking: Canonical-match preference was removed so that alternate matches can compete using geographic priority and population.

Fuzzy Performance: Approximate matching is more expensive than exact matching, motivating an indexed fuzzy-search structure.

19. FUTURE ENHANCEMENTS

19.1 Hash Table

A hash table can map normalized names directly to candidate records and greatly reduce average exact lookup time.

19.2 Trie

A trie can organize location names character by character and provide efficient prefix-based candidate generation.

19.3 BK-Tree

A BK-tree is a metric-tree structure suitable for approximate string matching using a distance such as Levenshtein distance. It can reduce the amount of scanning required for fuzzy search.

19.4 Multi-Word Location Detection

Future versions can support multi-word geographic names rather than relying primarily on single-word tokenization.

19.5 Context-Aware Disambiguation

Future work can incorporate country or sentence context so that ambiguous names can be resolved more accurately.

19.6 Other Extensions

- Larger lexical lists for common-word protection.
- Compact alternate-name storage.
- Automatic live database updates.
- Integration with map or geographic information systems.
- More efficient indexed geographic search.

20. CONCLUSION AND LEARNING OUTCOMES

20.1 Conclusion

The Geospatial Place Name Extractor & Entity Normalizer demonstrates how Data Structures and Algorithms can be applied to a practical geographic text-processing problem. The system uses a large GeoNames dataset, parses and filters geographic records, stores them in memory, performs exact and alternate-name matching, filters common words, ranks candidates and applies Levenshtein distance for reasonable spelling corrections.

The documented results demonstrate canonical recognition, alternate-name normalization and approximate spelling correction in a single sentence-processing pipeline. The project therefore provides a complete educational implementation connecting data preprocessing, structures, strings, searching, ranking and dynamic programming.

20.2 Learning Outcomes

- Practical file parsing of tab-separated data.
- Design and use of C structures and arrays.
- Dynamic memory allocation for large datasets.
- Case-insensitive string processing.
- Exact and alternate-name searching.
- Candidate ranking and deterministic tie-breaking.
- Dynamic programming through Levenshtein distance.
- Sentence reconstruction and output normalization.
- Testing and analysis of algorithmic limitations.

21. REFERENCES / BIBLIOGRAPHY

GeoNames geographic database and worldwide database extract documentation, as used by the project.

Project source documentation for the Geospatial Place Name Extractor & Entity Normalizer.

C programming language concepts: structures, arrays, strings, file processing and dynamic memory allocation.

Levenshtein edit-distance dynamic programming method used for approximate string matching.

Note: The references above are limited to the sources and concepts explicitly represented in the Geospatial project report.

22. APPENDIX

22.1 Representative Input and Output

Input:

iam travelling from mumbal to banglore

Detected:

mumbal → Mumbai

banglore → Bengaluru

Output:

iam travelling from Mumbai to Bengaluru

22.2 Algorithm Summary

Technique	Purpose
Custom tab parser	Preserve empty source fields
Linear exact search	Find canonical / alternate names
Feature priority	Rank geographic candidates
Population tie-break	Resolve equal-priority candidates
Common-word filter	Reduce false positives
Levenshtein distance	Correct spelling variations
Sentence reconstruction	Produce normalized full sentence

22.3 Final Demonstration

The integrated demonstration accepts a sentence, identifies geographic terms, normalizes alternate names and corrects reasonable spelling variations while retaining the rest of the sentence.

END OF PROJECT DOCUMENTATION